# Using Cloud Infrastructure as Code (IaC) to Deploy a Web Application

Cloud Computing – CSE456
Supervised by Dr. Wagdy Anis

Ahmed Mohamed Elsayed Tabbash (20P1076) Yaman Abdelhamid Elewa (2002078)
Ahmed Abdallah Nofull (22P0255) Ferass Ahmed Mostafa (23P0304)
*Faculty of Engineering, Ain Shams University*

***Abstract*—This paper presents the deployment of a web application on Amazon Web Services (AWS) using Infrastructure as Code (IaC) exclusively. The entire environment is expressed declaratively in a single AWS CloudFormation template, parameterised through an external JSON file, and provisioned without any manual configuration of individual resources. The resulting infrastructure comprises a Virtual Private Cloud spanning two Availability Zones, public and private subnets, an Internet Gateway, NAT Gateways, an Application Load Balancer, and an Auto-Scaling Group of web servers isolated in the private subnets. The solution was deployed, verified, subjected to a deliberate instance failure, and finally destroyed, all through the same template. The failure test showed that the Auto-Scaling Group restores capacity automatically and that the service remains reachable at an unchanged address, confirming that IaC can provision and reproduce a complete web application environment on demand.**

***Index Terms*—cloud, infrastructure as code, IaC, CloudFormation, AWS, auto-scaling, load balancing, web application**

## I. Introduction

Provisioning and maintaining cloud infrastructure by hand is slow, difficult to reproduce, and prone to human error. Infrastructure as Code (IaC) addresses this by describing the desired infrastructure in machine-readable files that can be versioned, reviewed and executed like any other source code [1]. The infrastructure then becomes a deterministic artefact rather than the residue of a sequence of manual actions in a web console. Among the benefits commonly attributed to the approach are lower cost, higher deployment speed, a reduced risk of human error, and the elimination of configuration drift [5].

Tooling for IaC broadly divides into model-driven and code-centric approaches, and empirical comparison of the two suggests that the choice of tool has a measurable effect on how effectively engineers can express and maintain an infrastructure definition [2]. AWS CloudFormation, the service used in this work, is code-centric and *declarative*: the engineer states the desired end state and the service determines the order in which resources must be created, updated or deleted in order to reach it [6]. Because the definition is a file, the same environment can be rebuilt identically in minutes, in any region, as many times as required.

Course project, Faculty of Engineering, Ain Shams University.

A comparable study by Eleraky *et al.* [4] used CloudFormation to construct an enterprise-oriented network, load-balanced website and database, and reported that IaC was able to meet those goals. The present project follows a similar path but places the web servers in private subnets behind NAT Gateways, and additionally verifies the behaviour of the environment under a deliberately induced server failure.

The objective of this project was to deploy a web application on AWS using IaC exclusively, and to verify not only that it deploys, but that it survives the failure of a server and can be torn down cleanly.

## II. Architecture and Methodology

### A. Planning

To provide fault tolerance, the architecture was deliberately distributed across two Availability Zones (AZs). Each zone contains one public and one private subnet.

- **Public tier.** Hosts the outward-facing resources: the Application Load Balancer and the NAT Gateways. Client traffic reaches this tier first.
- **Private tier.** Hosts the web servers. These have no route from the internet and are reachable only through the Load Balancer, which substantially reduces the attack surface.

### B. Parameter File

Configuration was separated from the declarative code by means of an external JSON parameter file. This keeps the template reusable: the same infrastructure can be deployed with different address ranges or instance sizes simply by supplying different values.

```
[
  { "ParameterKey": "EnvironmentName",
    "ParameterValue": "HA-Project" },
  { "ParameterKey": "VpcNetCIDR",
    "ParameterValue": "10.0.0.0/16" },
  { "ParameterKey": "PubSubnet01CIDR",
    "ParameterValue": "10.0.1.0/24" },
  { "ParameterKey": "PubSubnet02CIDR",
    "ParameterValue": "10.0.2.0/24" },
  { "ParameterKey": "PrivSubnet01CIDR",
    "ParameterValue": "10.0.3.0/24" },
  { "ParameterKey": "PrivSubnet02CIDR",
    "ParameterValue": "10.0.4.0/24" }
]
```

Two further parameters are declared in the template itself with default values: `InstanceType`, constrained by an `AllowedValues` list, and `LatestAmiId`, which is resolved at deployment time from the AWS Systems Manager public parameter store. The latter means the template does not hard-code a region-specific machine image and therefore deploys unchanged in any region.

## III. Implementation

### A. Virtual Private Cloud

The VPC is the container for every other resource. DNS support and DNS hostnames are enabled so that resources inside it can resolve one another by name.

```
MainVPC:
  Type: AWS::EC2::VPC
  Properties:
    CidrBlock: !Ref VpcNetCIDR
    EnableDnsSupport: true
    EnableDnsHostnames: true
```

### B. Subnets and Routing

Four subnets are created. The `!Select` and `!GetAZs` intrinsic functions distribute them across the region's Availability Zones without naming any zone explicitly, so the template remains portable.

```
PrivateSubnet1:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref MainVPC
    AvailabilityZone: !Select [0, !GetAZs ''']
    CidrBlock: !Ref PrivSubnet01CIDR
    MapPublicIpOnLaunch: false
```

A single route table is shared by both public subnets and directs `0.0.0.0/0` to the Internet Gateway. Each private subnet has its own route table directing outbound traffic to the NAT Gateway that lives in its own AZ, so that the loss of one zone cannot sever egress for the other [7].

### C. Gateways

One Internet Gateway attaches the VPC to the internet. Two NAT Gateways, each with a dedicated Elastic IP, allow the private web servers to make outbound connections – for example to install packages – while remaining unreachable from the outside.

### D. Auto-Scaling Group and Launch Template

The web servers are defined by an EC2 Launch Template, which supersedes the deprecated Launch Configuration. The Auto-Scaling Group (ASG) maintains between two and three instances and places them in the *private* subnets, registering each one with the Load Balancer's Target Group automatically [8].

```
ServersAutoScalingGroup:
  Type: AWS::AutoScaling::AutoScalingGroup
  DependsOn: [PrivRoute1, PrivRoute2]
  Properties:
    LaunchTemplate:
      LaunchTemplateId: !Ref HAProjectLaunchTemplate
      Version: !GetAtt
        HAProjectLaunchTemplate.LatestVersionNumber
    MinSize: '2'
    MaxSize: '3'
    VPCZoneIdentifier:
      - !Ref PrivateSubnet1
      - !Ref PrivateSubnet2
    TargetGroupARNs:
      - !Ref LBTargetGroup
```

The `DependsOn` clause is significant. Because the servers reach the package repositories only through the NAT Gateways, an instance launched before its private route exists has no path to the internet and its bootstrap script fails. Declaring the dependency explicitly forces CloudFormation to complete the routing before any instance is started. As a second safeguard, the bootstrap script retries the installation until the repositories become reachable.

```
UserData:
  Fn::Base64: |
    #!/bin/bash
    for i in $(seq 1 30); do
      yum install -y httpd && break
      sleep 10
    done
    mkdir -p /var/www/html
    echo '<html>...</html>' > /var/www/html/index.html
    systemctl enable httpd
    systemctl start httpd
```

### E. Load Balancer

An internet-facing Application Load Balancer is placed in the two public subnets. It presents a single, stable DNS name to clients and distributes requests across the healthy instances registered in its Target Group. The Target Group performs continuous HTTP health checks; an instance that stops responding is removed from rotation automatically [9].

```
LBTargetGroup:
  Type: AWS::ElasticLoadBalancingV2::TargetGroup
  Properties:
    HealthCheckIntervalSeconds: 10
    HealthCheckPath: /
    HealthCheckProtocol: HTTP
    HealthCheckTimeoutSeconds: 8
    HealthyThresholdCount: 2
    UnhealthyThresholdCount: 5
    Port: 80
    Protocol: HTTP
    VpcId: !Ref MainVPC
```

Security groups enforce the tiering: the Load Balancer accepts HTTP from the internet, while the servers accept HTTP *only* from the Load Balancer's security group.

## IV. Testing and Results

The stack was deployed through the AWS CloudFormation Management Console in the `us-east-1` region. The equivalent CLI invocation is:

```
aws cloudformation create-stack
  --stack-name HA-Project-Stack
  --region us-east-1
  --template-body file://infrastructure.yaml
  --parameters file://params.json
```

### A. Stack Creation

CloudFormation resolved the input parameters and created every resource in the correct dependency order, reaching the `CREATE_COMPLETE` state.

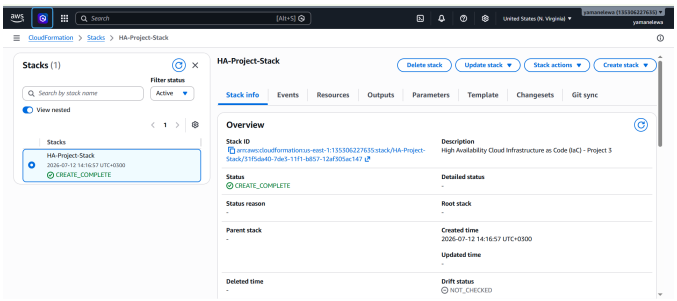


Fig. 1. Stack creation completed successfully.

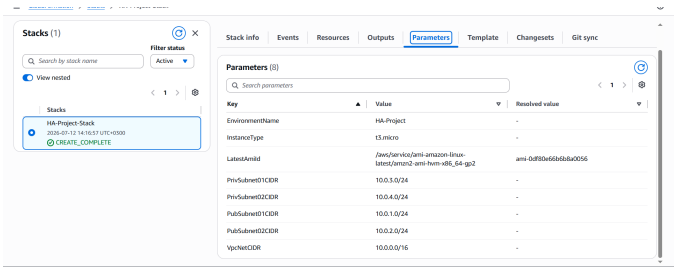


Fig. 2. The input parameters resolved by CloudFormation, including the automatically resolved Amazon Linux 2 machine image.

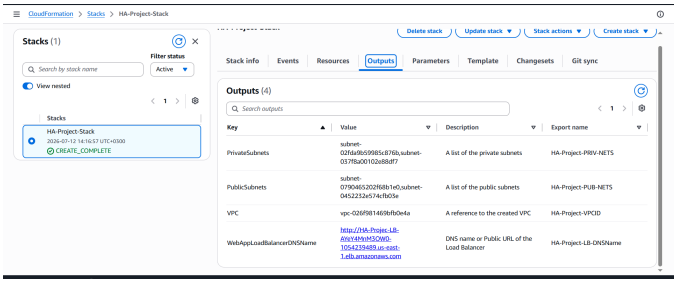


Fig. 3. The stack outputs, exporting the VPC, the subnets and the public URL of the Load Balancer.

## B. Verification of the Deployed Website

The `WebAppLoadBalancerDNSName` output gives the public URL of the Load Balancer. Browsing to it returned the page served by the EC2 instances in the private subnets, confirming the whole request path: client, to internet-facing Load Balancer in the public subnets, to a healthy web server in a private subnet.

## C. Auto-Scaling Recovery Test

To verify recovery from failure, one of the two running instances was terminated deliberately. The ASG observed that the group had fallen below its minimum size of two and launched a replacement in the same Availability Zone. The new instance registered itself with the Target Group and began serving traffic with no manual intervention.

Notably, the DNS name of the Load Balancer did not change at any point. Because the Load Balancer is a fixed entry point whose address is independent of the servers behind it, the site

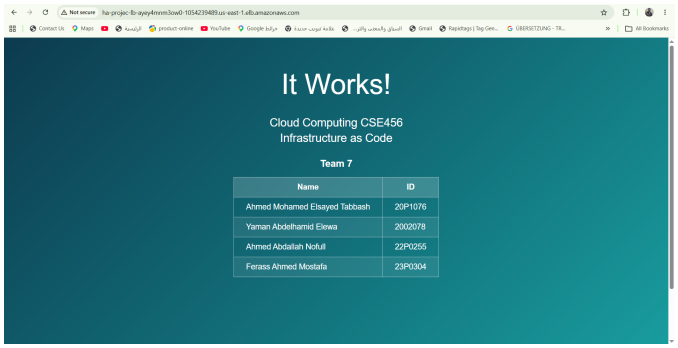


Fig. 4. The website served through the Load Balancer.

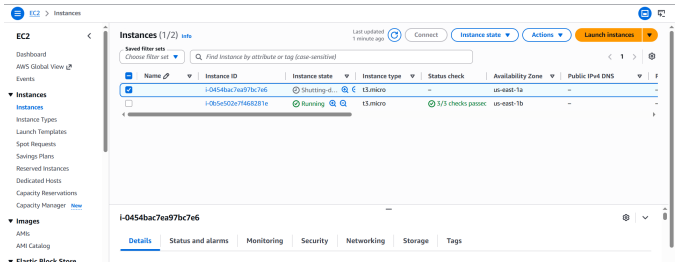


Fig. 5. The terminated instance shutting down while the surviving instance continues to serve traffic.

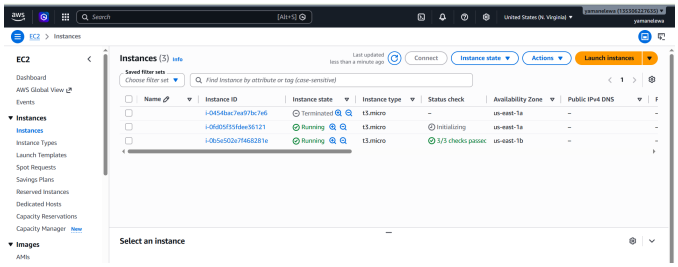


Fig. 6. A replacement instance launched automatically by the Auto-Scaling Group, restoring the minimum capacity of two.

remained reachable at exactly the same URL while the server that had been serving it was destroyed and rebuilt.

## D. Stack Deletion

Finally the environment was destroyed with a single delete operation. CloudFormation removed the resources in reverse dependency order and reported `DELETE_COMPLETE`, leaving no orphaned infrastructure and therefore no residual cost.

# V. Conclusion

This work showed that Infrastructure as Code can deploy a complete web application on AWS without a single manual configuration step. A declarative YAML template and a JSON parameter file were sufficient to create a two-zone network, isolate the compute tier in private subnets behind NAT Gateways, and distribute traffic through an Application Load Balancer.

The value of the approach was clearest in two places. First, the deliberate destruction of a running server was absorbed

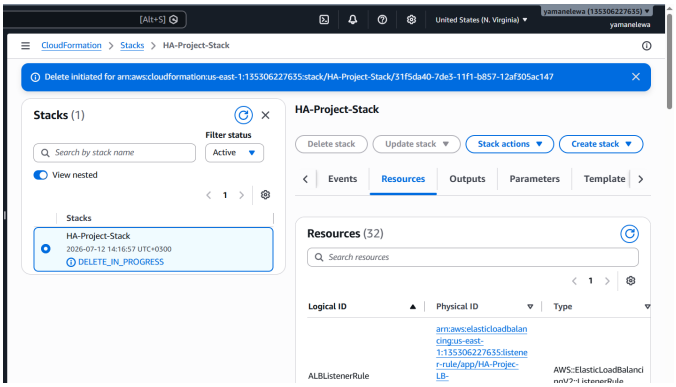


Fig. 7. Stack deletion in progress.

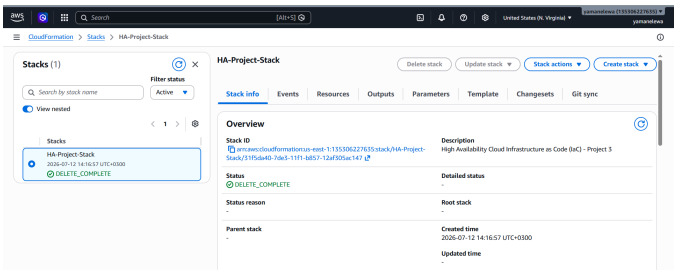


Fig. 8. Stack deletion completed successfully.

by the Auto-Scaling Group without human intervention and without any change to the address at which the service was reached. Second, the ordering defect uncovered during testing – instances being launched before their route to the internet existed – was fixed by a single declaration in the template, and the correction then applied identically to every future deployment. This is the central benefit of treating infrastructure as software: a defect is fixed once, in code, rather than repeatedly and inconsistently by hand.

## REFERENCES


[1] M. Artač, T. Borovssak, E. Di Nitto, M. Guerriero and D. A. Tamburri, "DevOps: Introducing Infrastructure-as-Code," in *Proc. IEEE/ACM 39th Int. Conf. on Software Engineering Companion (ICSE-C)*, 2017, pp. 497–498, doi: 10.1109/ICSE-C.2017.162.

[2] J. Sandobalín, E. Insfran and S. Abrahão, "On the Effectiveness of Tools to Support Infrastructure as Code: Model-Driven Versus Code-Centric," *IEEE Access*, vol. 8, pp. 17734–17761, 2020, doi: 10.1109/ACCESS.2020.2966597.

[3] G. Falazi *et al.*, "On Unifying the Compliance Management of Applications Based on IaC Automation," in *Proc. IEEE 19th Int. Conf. on Software Architecture Companion (ICSA-C)*, Honolulu, HI, USA, 2022, pp. 226–229, doi: 10.1109/ICSA-C54293.2022.00050.

[4] M. W. Eleraky, W. A. Aziz and J. N. Soliman, "Using Cloud Infrastructure as a Code (IaC) to Set Up Web Applications," Faculty of Engineering, Ain Shams University, 2023.

[5] C. BasuMallick, "What Is Infrastructure as Code? Meaning, Working, and Benefits," *Spiceworks*, Aug. 26, 2022. [Online]. Available: https://www.spiceworks.com/tech/cloud/articles/what-is-infrastructure-as-code/

[6] Amazon Web Services, "AWS CloudFormation User Guide." [Online]. Available: https://docs.aws.amazon.com/cloudformation/

[7] Amazon Web Services, "Amazon Virtual Private Cloud User Guide." [Online]. Available: https://docs.aws.amazon.com/vpc/

[8] Amazon Web Services, "Amazon EC2 Auto Scaling User Guide." [Online]. Available: https://docs.aws.amazon.com/autoscaling/ec2/userguide/

[9] Amazon Web Services, "Application Load Balancer User Guide." [Online]. Available: https://docs.aws.amazon.com/elasticloadbalancing/latest/application/